

Project 5: Analysing Adversarial Attacks on Network Data Classifiers

Clarifications for this project can be asked to **Gabriele Ciravegna**: gabriele.ciravegna@polito.it

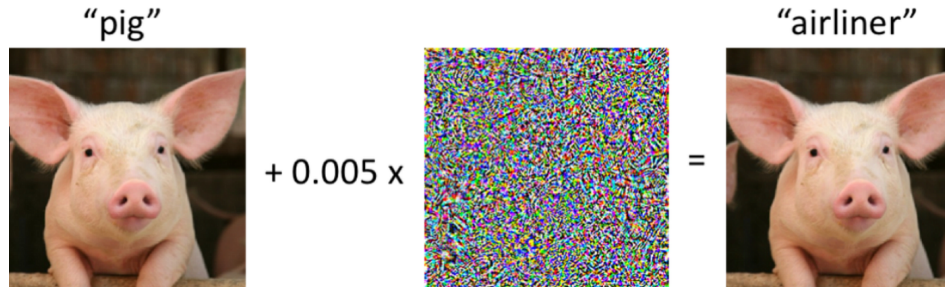


Figure 1: Examples of an Adversarial Attack on an image classifier. The same concept also applies for network data.

1 Introduction

Guaranteeing the security of Machine Learning (ML) classifiers is a crucial priority of all institutions employing automatic decision-making systems, to protect their businesses against cyberattacks and fraudulent attempts. **Adversarial attacks** are novel techniques that, have been proven to be effective to **fool image classification models**, as shown in Figure 1. However they can also be applied to tabular data, such as network data. Adversarial attacks aim at producing adversarial examples, in other words, slightly modified inputs that induce the Artificial Intelligence (AI) system to return incorrect outputs that are advantageous for the attacker. To illustrate the threat of adversarial attacks in a network data context, consider a scenario where an application detection system is used to identify specific applications based on network traffic. The system relies on a machine learning model to classify network flows based on features such as packet size, inter-arrival times, and flag counts. The model flags certain traffic as belonging to specific applications. However, an attacker is determined to bypass the detection system by subtly altering the network traffic to mislead the model. The key for this attack to succeed is its imperceptibility: the modified traffic should remain credible and relevant, in coherence with the model's prediction. As shown in Figure 1, an **adversarial attack must create adversarial samples** x_{adv} that are sufficiently **close to the real data** x , i.e., $|x - x_{adv}|_2 < \epsilon$, where ϵ is the maximum distance to which an attack is considered credible.

This assignment challenges students to explore the impact of adversarial attacks on tabular data classifiers, specifically within the context of detecting applications using network traffic data. This is a common task used in many contexts, such as network security and traffic management. The task involves a step-by-step investigation structured around essential components.

2 Data acquisition and exploration

2.1 Dataset Acquisition

To acquire the *IP Network Traffic Flows Labeled with 75 Apps* dataset, you can download it from here its kaggle repository available at <https://www.kaggle.com/datasets/jsrojas/ip-network-traffic-flows-labeled-with-87-apps>. The size of the dataset is quite high, therefore you are required to work with

a reduced version of the dataset that only contains the following apps: YOUTUBE, AMAZON, GMAIL, WINDOWS_UPDATE, SKYPE, FACEBOOK, DROPBOX, YAHOO, TWITTER, MSN.

2.2 Data Preprocessing

As a preprocessing step, **convert features if needed**, e.g., by means of a one-hot encoding, and **apply a normalization** to map all features between $[0, 1]$. Explain your preprocessing step according to the type of data and the ML models that you will employ. Clearly define categories or classes representing the traffic application. Additionally, reserve 20% of the dataset for subsequent testing and assessment of the robustness of the models.

2.3 Exploratory Data Analysis

Delve into the dataset's patterns using various visualization techniques, such as histograms or box plots, scatter plots and correlation matrices of the most interesting features. Provide insights into underlying patterns and, most importantly, identify features that may influence traffic application classification.

2.4 Data Visualization

Apply dimensionality reduction techniques such as t-Distributed Stochastic Neighbor Embedding (t-SNE) or Principal Component Analysis (PCA) to reduce and visualize the data.

2.5 Unsupervised Data Analysis

Employ a few unsupervised clustering algorithms studied during the course to group instances based on similar characteristics. Assess the quality of clusters and characterize them for the most common features. Also try to assess their potential implications by controlling the purity with respect to application label.

3 Supervised Data analysis

3.1 Classifier Selection

Experiment with a range of classifiers suitable for binary classification tasks. Select a minimum of three classifiers studied in the course (e.g., Decision Trees, Support Vector Machines, Neural Networks). At least one has to be a Neural Network for the following adversarial evaluation.

3.2 Cross Validation

Implement a rigorous cross-validation process involving training, evaluating and tuning of the hyperparameters to get the best validation accuracy. Each classifier's performance should be summarized with a brief description of the optimized hyperparameters and a rationale for observed outcomes.

3.3 Classifier Evaluation

Evaluate the best selected classifier (the one obtaining the highest classification accuracy on the validation set) on the testing set. Utilize metrics like accuracy, precision, recall, F1-score, and area under the ROC curve (AUC-ROC) to assess the classifier's effectiveness in traffic application classification.

4 Adversarial Attacks

4.1 Random Noise

Apply random noise to the test data and measure its impact on classifier accuracy. Experiment with varying intensities of noise ϵ (clip the noise to have ϵ Euclidean norm) to measure the degradation

in classifier performance. This metric is also called Robust Accuracy, i.e., the accuracy of a classifier when facing adversarial data.

4.2 Feature specific noise

Conduct a detailed study on the effect of random noise on individual features. Explore the classifier's sensitivity to specific features and identify the most vulnerable one. Determine the minimum distance ϵ required to frequently deceive the classifier. Discuss the implications of these adversarial attacks on the models, based on the example provided in the Introduction.

4.3 Adversarial Attack with ART

The Adversarial Robustness Toolbox (ART)¹ is a Python library for Machine Learning Security. ART provides tools for the automatic creation of adversarial attacks, based on advanced optimization techniques. Following the example in https://github.com/Trusted-AI/adversarial-robustness-toolbox/blob/main/examples/get_started_scikit_learn.py (works with scikit-learn, checks the one about Pytorch if you are using that instead), evaluate the adversarial attack against the neural network classifier at different distances (epsilon parameter). Is the distance similar to the one you obtained? Which features are affected more on average? Test at least two attacks implemented in the library (e.g., FGSM and PGD)

4.4 Countermeasure Exploration

Explore potential countermeasures to mitigate the impact of adversarial attacks on the traffic classification model. For instance, you can consider adversarial training: which consists in training a new classifier on both the standard data and the adversarial data. Re-evaluate the classifier equipped with countermeasure on both clean and new adversarial data to check both clean and robust accuracy.

References:

While external references are not strictly necessary for this assignment (except for ART), any utilized resources, such as course materials, should be appropriately cited within the document.

¹<https://github.com/Trusted-AI/adversarial-robustness-toolbox>